

Time-Series Autoencoder for Suspicious Financial Transaction Periods

Master Documentation

Main business question

Can a time-series autoencoder learn normal transaction behaviour over time and detect suspicious transaction periods?

Document section	What it covers
Project summary	Notebook objective, business problem, dataset setup, and final results
Theory A-Z	Time series, sliding windows, LSTM, sequence reconstruction error, and thresholding
Data and variable flow	How raw transaction rows become time-step summaries and sliding windows
Model workflow	LSTM autoencoder architecture, training, reconstruction error, and anomaly threshold
Results and BI story	Training loss, suspicious windows, fraud count comparison, KPI chart, and interpretation
Limitations and next steps	Why the result is useful, what it does not prove, and how to improve it

1. Executive Summary

This document summarizes the time-series autoencoder notebook built on the PaySim financial transaction dataset. The goal was to move beyond row-level anomaly detection and detect suspicious transaction periods using time-window sequence modelling.

The previous anomaly detection notebook studied individual transactions. This notebook aggregates transactions by time step, creates sliding windows, trains an LSTM autoencoder, and uses sequence reconstruction error as an anomaly score.

Key finding

The LSTM autoencoder flagged 14 out of 89 test windows as suspicious. These windows had higher reconstruction error, but they did not have higher average fraud rate. The model detected unusual transaction-period behaviour, not direct fraud labels.

Metric	Value	Meaning
Total test windows	89	Number of time windows analysed in the test period
Predicted normal windows	75	Windows below the anomaly threshold
Predicted suspicious windows	14	Windows above the anomaly threshold
Suspicious window share	15.73%	Share of test windows flagged for investigation
Average error - normal windows	0.626308	Average anomaly score for normal windows
Average error - suspicious windows	0.902025	Average anomaly score for suspicious windows
Total fraud count - normal windows	836	Fraud count observed in normal windows
Total fraud count - suspicious windows	146	Fraud count observed in suspicious windows
Average fraud rate - normal windows	0.689615	Average fraud intensity in normal windows
Average fraud rate - suspicious windows	0.364280	Average fraud intensity in suspicious windows

2. Theory A-Z in Simple Language

2.1 Time series

A time series is data arranged in time order. In this notebook, the PaySim step column acts as the time unit. Instead of looking at one transaction at a time, we study how transaction behaviour changes from one step to the next.

2.2 Time step

A time step is one period in time. Each step contains many transactions. We aggregate those transactions so that one row describes one time step.

2.3 Aggregation

Aggregation means summarising many transaction rows into one time-step row. For each step, we calculate transaction count, total amount, average amount, maximum amount, transaction type counts, transaction type shares, fraud count, and fraud rate.

2.4 Sliding window

A sliding window converts ordered time-step data into sequences. With a window size of 24, window 1 contains steps 1-24, window 2 contains steps 2-25, and so on. Each window becomes one model input sample.

2.5 LSTM autoencoder

An LSTM autoencoder is a neural network that reconstructs a full time sequence. The encoder reads the sequence and compresses it. The decoder rebuilds the sequence. If the model reconstructs a window poorly, that window may contain unusual behaviour.

2.6 Sequence reconstruction error

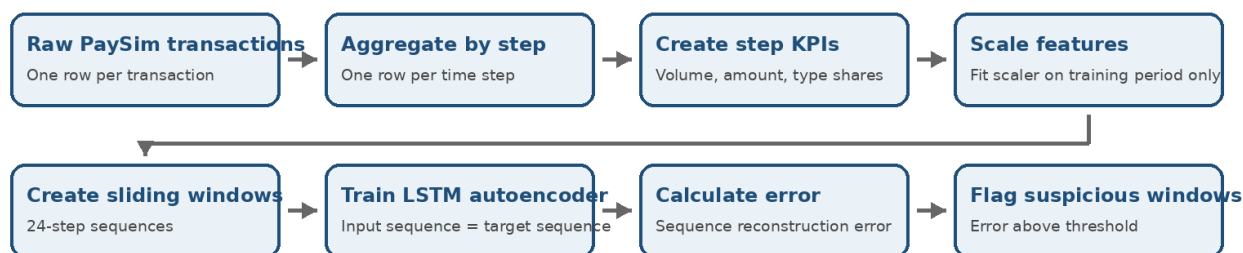
Sequence reconstruction error measures how different the reconstructed sequence is from the original sequence. Low error means the window looked familiar to the model. High error means the model struggled to rebuild the window.

Concept	Simple meaning	Notebook use
Time series	Data ordered by time	PaySim steps represent transaction periods
Time step	One time period	Each step becomes one summary row
Aggregation	Summarise many rows into one row	Transaction KPIs by step
Sliding window	Moving sequence of steps	24-step windows for LSTM input
LSTM autoencoder	Reconstructs sequences	Learns normal time-window behaviour
Reconstruction error	How badly the model rebuilt the window	Anomaly score
Threshold	Cutoff for suspicious windows	95th percentile of validation error

3. End-to-End Workflow

The workflow starts with raw PaySim transaction rows and ends with suspicious time windows. The fraud columns are kept outside the model and used only for evaluation and interpretation.

Time-Series Autoencoder Workflow



Business message: the model identifies unusual transaction periods, not direct fraud labels.

Figure 1. Time-series autoencoder workflow from raw transactions to suspicious windows.

4. Data Preparation and Feature Engineering

4.1 Raw dataset

The raw PaySim dataset contains one row per transaction. Important columns include step, type, amount, sender and receiver balances, isFraud, and isFlaggedFraud. For the time-series notebook, the step column is the most important because it allows us to build ordered transaction periods.

4.2 Step-level aggregation

The raw transaction rows were grouped by step. This changed the dataset structure from one row per transaction to one row per time step.

Created feature	Meaning	Used as model input?
transaction_count	Number of transactions in the time step	Yes
total_amount	Total amount moved in the time step	Yes
average_amount	Average transaction amount	Yes
max_amount	Largest transaction amount	Yes
cash_in_share	Share of CASH_IN transactions	Yes
cash_out_share	Share of CASH_OUT transactions	Yes
debit_share	Share of DEBIT transactions	Yes
payment_share	Share of PAYMENT transactions	Yes
transfer_share	Share of TRANSFER transactions	Yes
fraud_count	Number of fraud transactions in the step	No - evaluation only
fraud_rate	Fraud share in the step	No - evaluation only

4.3 Why fraud columns are excluded

The model must learn transaction behaviour patterns, not fraud labels. Fraud_count and fraud_rate are excluded from the model input so that the LSTM autoencoder cannot directly learn the label. They are used later to interpret whether high-error windows align with fraud activity.

5. Variable, Method, and Data Flow

Variable / object	Created from	Purpose
transactions	CSV file	Raw PaySim transaction data
step_amount_summary	transactions grouped by step	Amount KPIs and fraud count per step
type_counts	pivot table by step and type	Transaction type counts per step
step_data	merged summaries	Final step-level time-series dataset
time_series_features	selected behaviour columns	Model input features
time_series_evaluation	step, fraud_count, fraud_rate	Evaluation and business interpretation
train_scaled / val_scaled / test_scaled	StandardScaler output	Scaled time-step features
train_windows / val_windows / test_windows	sliding-window function	3D LSTM model input
lstm_autoencoder	Keras model	Sequence reconstruction model
test_window_results	test evaluation + reconstruction error	Final anomaly analysis table

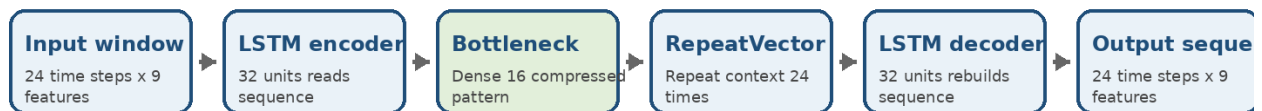
5.1 Shape logic

The important shape change is from a 2D time-step table to a 3D LSTM input array.

Stage	Shape logic	Meaning
Step-level features	time steps x features	One row per time step
Sliding windows	windows x window_size x features	One sample is a sequence
Model output	windows x window_size x features	Output shape matches input shape
Reconstruction error	windows	One anomaly score per time window

6. LSTM Autoencoder Model

LSTM Autoencoder Architecture



Training target: the model reconstructs the same sequence it receives. Fraud labels are not used as targets.

Anomaly score: mean squared reconstruction error across all time steps and features in one window.

Figure 2. LSTM autoencoder architecture used for sequence reconstruction.

The model receives one sequence window at a time. The encoder compresses the sequence into a small representation. The decoder reconstructs the full window. The model is trained with input = target, so fraud labels are not used during training.

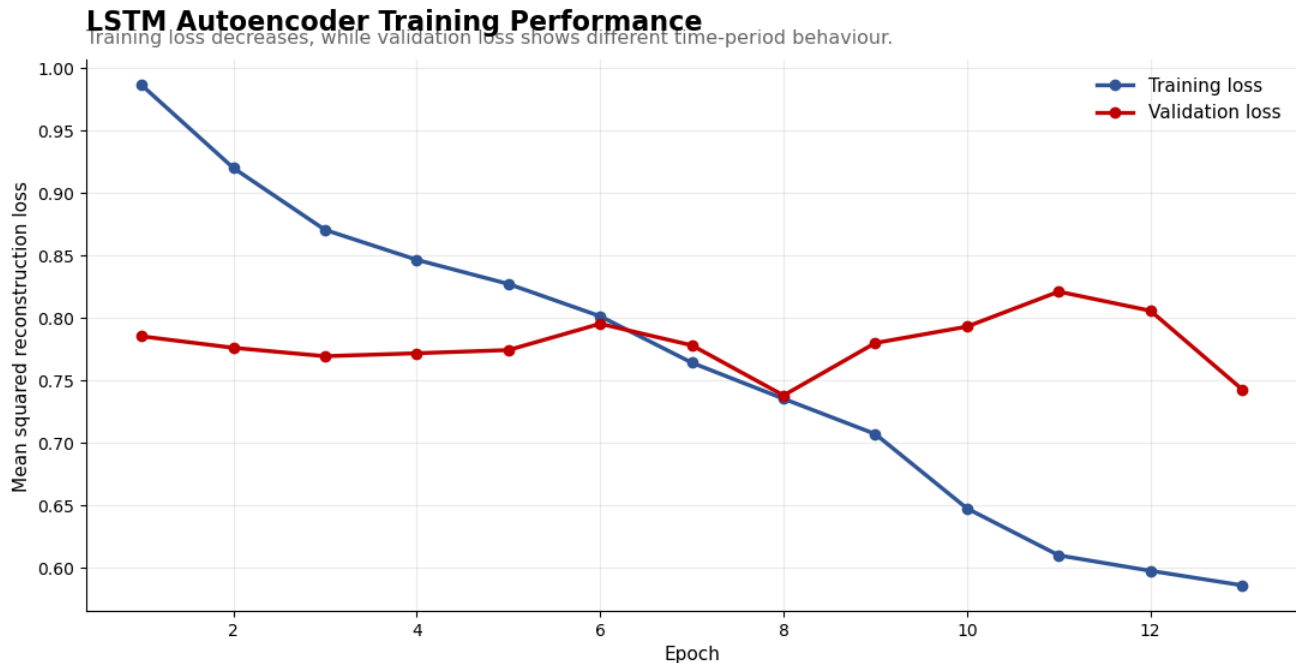
Layer / step	Configuration	Purpose
Input	window size x number of features	Receives one time-window sequence
LSTM encoder	32 units	Reads the ordered sequence
Bottleneck	Dense 16	Compressed representation of the window
RepeatVector	number of time steps	Repeats bottleneck for decoding
LSTM decoder	32 units, return_sequences=True	Rebuilds the sequence
TimeDistributed Dense	number of features	Reconstructs features at each time step
Loss	Mean squared error	Measures reconstruction quality

6.1 Training result

Metric	Value	Interpretation
Total epochs trained	13	EarlyStopping stopped training after validation loss stopped improving
Final training loss	0.585413	Model learned to reconstruct training windows
Final validation loss	0.742436	Validation behaviour differed from training period
Validation pattern	Unstable / higher than training loss	Possible behaviour shift across time periods

7. Results and Visual Interpretation

7.1 LSTM training performance



Business note: A validation gap may indicate behaviour shift between earlier and later transaction periods.

Figure 3. LSTM autoencoder training and validation loss.

Training loss decreased steadily, showing that the model learned to reconstruct training windows. Validation loss was higher and less stable, suggesting that the validation period contained behaviour different from the earlier training period.

7.2 Anomaly threshold

The anomaly threshold was calculated from validation reconstruction error. The selected threshold was the 95th percentile of validation error.

Validation error metric	Value
Mean	0.737601
Median	0.746178
95th percentile threshold	0.877242

7.3 Reconstruction error over time

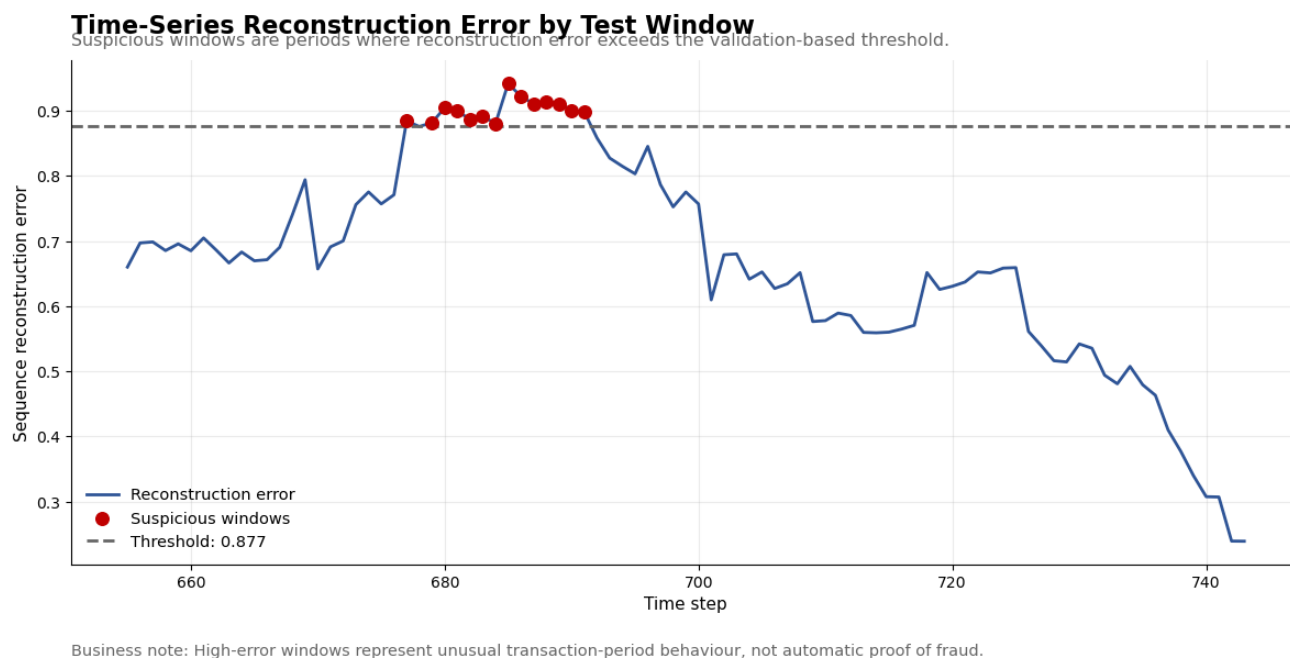


Figure 4. Reconstruction error over test windows with suspicious windows highlighted.

Suspicious windows appeared mainly around steps 678-691. These windows crossed the threshold and were therefore treated as unusual transaction periods. After that range, reconstruction error gradually declined.

7.4 Reconstruction error vs fraud count

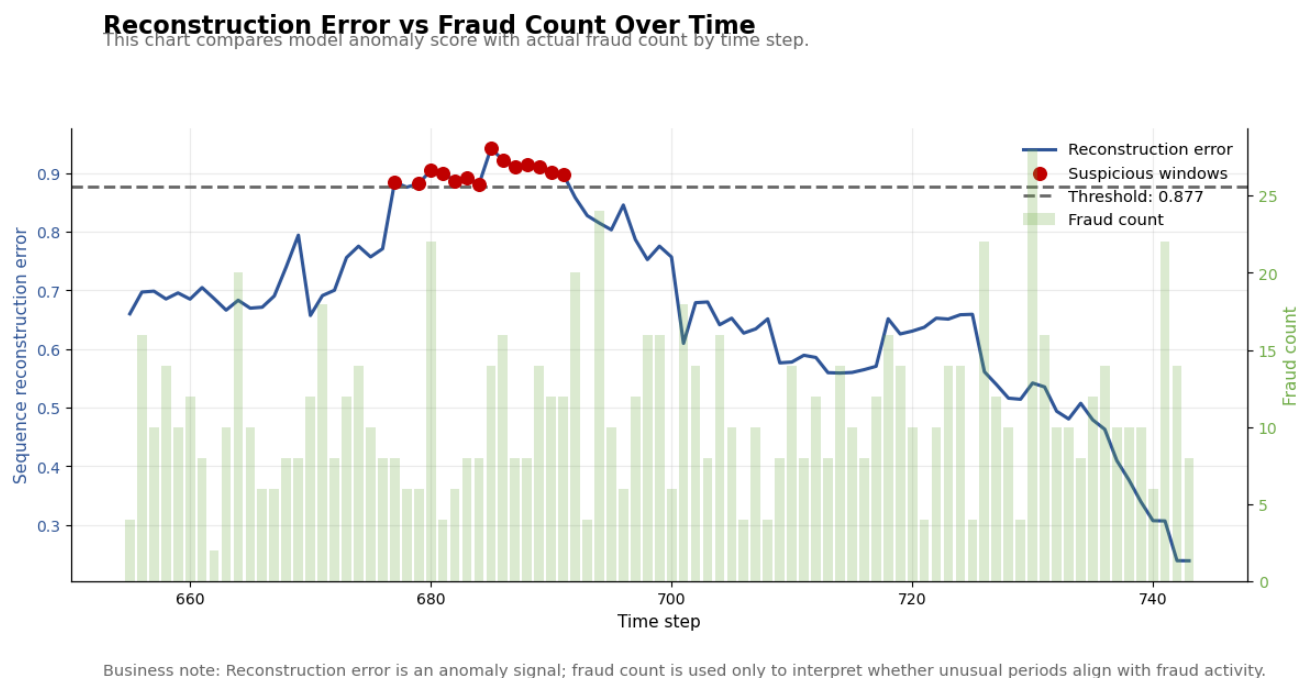


Figure 5. Reconstruction error compared with actual fraud count over time.

The suspicious windows did not perfectly align with the highest fraud counts. This is expected because `fraud_count` was not used as a model input. The model detected unusual behaviour patterns, not direct fraud labels.

7.5 KPI comparison chart

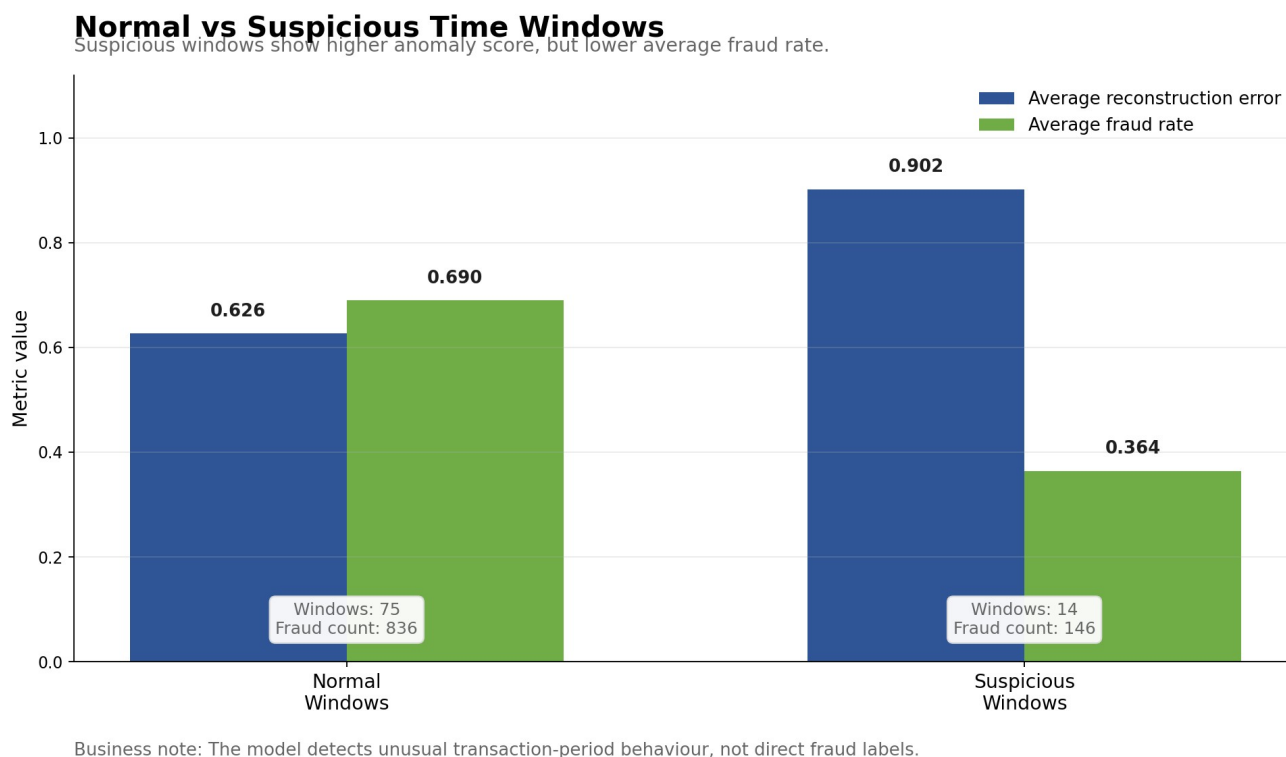


Figure 6. Normal vs suspicious time-window KPI comparison.

Suspicious windows had higher average reconstruction error, confirming that the LSTM autoencoder found them unusual. However, suspicious windows had lower average fraud rate than normal windows. This is the most important business interpretation of the notebook.

8. Business Interpretation

Main business takeaway

The model is useful as an unusual-period monitoring signal, but it should not be interpreted as direct fraud classification.

The LSTM autoencoder detected periods where transaction behaviour looked different from the learned pattern. This is valuable for monitoring and investigation prioritisation. However, the suspicious windows were not the fraud-heavy periods by average fraud rate.

Observation	Business meaning
14 out of 89 windows were suspicious	The model narrowed the review period to a smaller subset of time windows
Suspicious windows had higher reconstruction error	The model found clear unusual behaviour in those periods
Suspicious windows had lower average fraud rate	The anomaly score does not directly equal fraud intensity
Suspicious period around steps 678-691	This period should be reviewed for behavioural shift
Fraud appears across many steps	Fraud activity is distributed and not only concentrated in anomalous windows

9. Limitations

- The model detects unusual time-window behaviour, not direct fraud labels.
- Fraud_count and fraud_rate were excluded from model inputs, so alignment with fraud is indirect.
- Chronological splitting can produce train-validation-test behaviour shifts, especially if later periods differ strongly from earlier periods.
- The 95th percentile threshold is explainable, but it is not automatically optimal.
- The current setup uses step-level aggregation, so individual transaction-level detail is lost.
- Some fraud_rate values can be high when a time step has very few transactions, so fraud rate must be interpreted carefully.

10. Recommended Next Steps

Next step	Why it helps
Try threshold comparison	Understand how suspicious-window volume changes at 90%, 95%, 97.5%, and 99% thresholds
Use window-level fraud features for evaluation	Compare suspicious windows with total fraud inside the full window, not only the final step
Add more step-level features	Include balance-change behaviour, cash-out amount share, transfer amount share, and amount volatility
Compare with simple baselines	Check whether LSTM adds value over rolling z-score or Isolation Forest
Create a monitoring dashboard	Track reconstruction error, threshold, suspicious windows, fraud count, and transaction mix
Tune model architecture	Adjust LSTM units, bottleneck size, dropout, and window size

11. Interview-Ready Explanation

I converted the transaction-level PaySim dataset into a time-series problem by aggregating transactions by time step. For each step, I created behaviour KPIs such as transaction count, total amount, average amount, maximum amount, and transaction type shares. I kept fraud_count and fraud_rate separately only for evaluation.

Then I scaled the behaviour features and created sliding windows with 24 consecutive time steps per sample. I trained an LSTM autoencoder to reconstruct those windows. Since this is an autoencoder, the input and target were the same sequence. Fraud labels were not used for training.

After training, I calculated sequence reconstruction error for each test window. I used the 95th percentile of validation reconstruction error as the anomaly threshold. Windows above that threshold were flagged as suspicious.

The model flagged 14 out of 89 test windows. These suspicious windows had higher reconstruction error than normal windows, which confirms that the model detected unusual transaction-period behaviour. However, suspicious windows did not have higher average fraud rate, so I interpreted the model as a behaviour monitoring signal rather than a direct fraud classifier.

12. Final Conclusion

This notebook successfully demonstrated a time-series autoencoder workflow using the PaySim dataset. The key achievement was transforming raw transaction rows into time-step summaries, creating sliding windows, training an LSTM autoencoder, and using reconstruction error to identify unusual transaction periods.

The model found suspicious windows mainly around steps 678-691. These windows had higher sequence reconstruction error than the rest of the test period. However, they did not have higher average fraud rate. This result is important because it shows the difference between anomaly detection and fraud classification.

Final statement

The LSTM autoencoder is useful for identifying unusual transaction-period behaviour. It can support fraud monitoring and investigation, but it should be combined with business rules, fraud labels, and analyst review before being used as a fraud detection decision system.

13. Glossary

Term	Meaning
Time step	One period in the ordered transaction timeline
Aggregation	Summarising many transaction rows into one time-step row
Sliding window	A moving group of consecutive time steps
Sequence	A time-ordered window used as one model sample
LSTM	Neural network layer designed for ordered sequence data
Autoencoder	Model that reconstructs its own input
Bottleneck	Compressed representation learned by the model
Reconstruction error	Difference between original and reconstructed input
Anomaly threshold	Cutoff used to flag high-error windows
Suspicious window	A time window with reconstruction error above the threshold